Design And Analysis of Algorithms
Lecture 20: Streaming

April 4, 2018

This lecture, we'll take a look at a different model of computation, which is (like Johnson-Lindenstrauss last lecture) focused on doing computations where space might be one constraint we care about "more than usual", meaning we might not have space to even store our entire input. The model we'll think about is the streaming model, where there is some universe U of elements, and on each day a new element $a_t \in U$ arrives, and on any day we might be asked to answer $f(a_{[1:t]})$ where $a_{[1:t]} = (a_1, \ldots, a_t)$ for some function f, and ideally we'd like not to store all of $a_{[1:t]}$. For arbitrary functions this won't be possible, but here are some examples of what we might hope to compute while using less space:

Sums or averages Here, we consider $f(a_{[1:t]}) = \sum_{t' \leq t} a_{t'}$, the function which computes sums. How many bits do we need to be able to answer sum queries every day? If each value $a_{t'} \leq 2^b$, then we simply need to store the sum, a number which is at most $2^b \cdot T$, which requires only $\log(2^b \cdot T) = O(b + \log(T))$ bits.

Maximum Value Seen So Far Suppose instead we'd like to keep track of the largest element seen so far in the stream. Then, we need to only keep b bits, where the values are guaranteed to be at most 2^b.

Median ... this seems somewhat harder. It seems like I can force my algorithm to remember the entire stream if it guarantees this for arbitrary length-substreams, if one doesn't use approximation. So do the next couple of examples:

Number of distinct elements

Mode/Approximate mode/"Heavy hitting" elements

1 Streams as Vectors

It will be useful to abstract away the "ordering" of the elements we've seen through round t (note: all the examples we suggested in the previous section didn't depend on when elements arrived). Let x_i^t represent the *number of times element $i \in U$ has been added in the first t terms of the stream.* So, we define the vectorized version of a stream $a_{[1:t]}$ to be:

$$\mathbf{x^t} = (x_1^t, \ldots, x_{|U|}^t)$$

which is just the "histogram" of the different elements' arrivals.

When one considers this abstraction, we might be interested in computing various norms of $\mathbf{x^t}$: for example $||\mathbf{x^t}||_0$, the number of distinct elements which have arrived through round t, or $||\mathbf{x^t}||_2$, the "surprise number" of the stream, which measures the spread of the distribution of elements. In general we'll define $F_p(\mathbf{x^t}) = ||\mathbf{x^t}||_p$, and could be interested in measuring this for any $p \geq 1$.

We would also like to assume we can *remove* elements via an update from the stream, rather than just add an element. So, we now assume $a^t \in \{\text{add, delete}\} \times U$, where $a_t = (\text{del}, e)$ means that $\mathbf{x}_e^{t+1} = \mathbf{x}_e^t - 1$ and $a_t = (\text{add}, e)$ means that $\mathbf{x}_e^{t+1} = \mathbf{x}_e^t + 1$.

2 Computing F_p, the pth "moment" of a stream

2.1 Computing F_2

Consider the following algorithm:

Algorithm 1: F_2-est

```
Pick h a random hash function from U to {±1};
Let C ← 0;
for t = 1 to T do
    If a_t = (del, i), set C ← C − h(i);
    If a_t = (add, i), set C ← C + h(i);
    To answer F_2(a_[1:t]), return C^2;
end
```

This algorithm uses a hash function, something we haven't used explicitly in the class until now. There are a couple of useful facts/properties about hash functions that we'll need, so we'll introduce those next:

Definition 1 (k-wise independent/universal)**.** *A distribution over functions $h : U \to R$ is said to be k-universal, or k-independent, if for any k distinct elements $i_1, \ldots, i_k \in U$, and any possible hash values $r_1, \ldots, r_k \in R$, we have that*

$$\mathbb{P}[\wedge_{j \in [k]} h(i_j) = r_j] = \frac{1}{|R|^k}.$$

This definition implies that the probability of collisions amongst sets of size at most k "looks random", and that each element is hashed to each hash value with uniform probability. It's not as strong as requiring full independence across all $|U|$ inputs if k is reasonably small. For our purposes, it will suffice to consider 4-wise independent hash function distributions. Since we're in the business of saving space, it's reasonable to ask how much space is required to store a k-universal hash function (since, if we end up storing $|U|$ bits, we may as well give up on this whole streaming business).

Remark 1. *To store a k-universal hash function, it is necessary and sufficient to use $O(k \log |U|)$ bits of memory.*

So, if we need less than full independence of our hash function, this approach might save us some real space in terms of $|U|$.

Theorem 1. *Algorithm 1 uses $O(\log |U|) + \log(T))$ space, $\mathbb{E}[F_2\text{-est}] = F_2(\mathbf{x^t})$ if h is a 4-universal function.*

Proof. The space guarantee follows from the fact that the algorithm merely stores a $4-$universal hash function and the sum of T bits which are plus or minus 1, which can be represented using $\log(T)$ bits. Let $x_i = \sum_{t' \leq t} \mathbb{I}[a_t = (\text{add}, i)] - \sum_{t' \leq t} \mathbb{I}[a_t = (\text{del}, i)]$ denote the cumulative count of element i prior to round i.

Then, we know that

$$\begin{aligned}
\mathbb{E}[F_2\text{-est}] &= \mathbb{E}[C^2] \\
&= \mathbb{E}[\sum_{i,j\le|U|} x_i x_j h(i)h(j)] \\
&= \sum_{i,j\le|U|} x_i x_j \mathbb{E}[h(i)h(j)] \\
&= \sum_{i\le|U|} x_i^2 \mathbb{E}[h(i)^2] + \sum_{i\neq j\in U} x_i x_j \mathbb{E}[h(i)h(j)] \\
&= \sum_{i\le|U|} x_i^2
\end{aligned}$$

where the last inequality follows from $h(i)^2 = 1$, and $\mathbb{E}[h(i)h(j)] = 0$ for distinct i since h is 4-universal. □

To compute the variance of C^2, we start by computing

$$\begin{aligned}
\mathbb{E}[C^4] &= \mathbb{E}[\sum_{p,q,r,s\le|U|} x_p x_q x_r x_s h(p)h(q)h(r)h(s)] \\
&= \mathbb{E}[\sum_{p\le|U|} x_p^4 h^4(p)] + 6\mathbb{E}[\sum_{p<q} x_p^2 x_q^2 h^2(p)h^2(q)] \\
&= \sum_{p\le|U|} x_p^4 + 6\mathbb{E}[\sum_{p<q} x_p^2 x_q^2 h^2(p)h^2(q)] \qquad = \sum_{p\le|U|} x_p^4 + 6\mathbb{E}[\sum_{p<q} x_p^2 x_q^2]
\end{aligned}$$

where the second equality follows from the fact that when there is any one of p, q, r, or s which is different from the other three values, the expectation of the products of the hashed values is 0 (since $\mathbb{E}[h(p)h(q)h(r)h(s)] = E[h(p)]E[h(q)h(r)h(s)] = 0$ if $p \notin \{q, r, s\}$ by 4-wise independence).

Then, we have that

$$\begin{aligned}
Var(C^2) &= \mathbb{E}[(C^2)^2] - \mathbb{E}[C^2]^2 \\
&= \sum_{p\le|U|} x_p^4 + 6\mathbb{E}[\sum_{p<q} x_p^2 x_q^2] - (\sum_{i\in U} x_i^2)^2 \\
&= 4\mathbb{E}[\sum_{p<q} x_p^2 x_q^2] \le 2\mathbb{E}[C^2]^2
\end{aligned}$$

which combined with Chebyshev says

$$\mathbb{P}[|C^2 - \mathbb{E}[C^2]| \ge \epsilon\mathbb{E}[C^2]] \le \frac{Var(C^2)}{(\epsilon\mathbb{E}[C^2])^2} \le \frac{2}{\epsilon^2}$$

which isn't impressive at all for $\epsilon < 1$, since this is a vacuous statement.

If one wants to improve the variance of this estimator, one way to do that is to *run b independent copies* of this algorithm, average together the copies of C, then square: that increases the space complexity by a factor of b, while decreasing the variance by a factor of $\frac{1}{b}$. Then, we know that

$$\mathbb{P}[|C^2 - \mathbb{E}[C^2]| \ge \epsilon\mathbb{E}[C^2]] \le \frac{Var(C^2)}{(\epsilon\mathbb{E}[C^2])^2} \le \frac{2}{b\epsilon^2}$$

which, for appropriate b, will at least be less than 1.

Remark 2. **?** *introduced this algorithm for estimating F_0, F_1, and F_2, while showing one needs space polynomial in $|U|$ to estimate higher moments. This idea was extended to estimating F_p with $2p$-universal hash functions by Ganguly [2004]; the variance gets a lot nastier for larger p.*

2.2 An aside: computing approximate matrix products

The ideas presented in the previous section can be used to speed up (approximate) matrix multiplication. Rather than computing AB for two $n \times n$ real-valued matrices, we can think of computing AS^TSB for some appropriate matrix S, and if S is "wide and short" enough, this computation will be faster than multiplying A and B.

Define $S_{ij} = \frac{1}{\sqrt{k}} h_i(j)$, the hash value of j using the ith random hash function in a collection. Then, S will be of "width" n, and of some height we will choose k to guarantee some approximation with respect to the following norm:

Definition 2. *The* Frobenius norm*, $||\cdot||_F$, of a matrix M, is defined to be $||M||_F = \sqrt{\sum_{i,j}(M_{i,j}^2)}$.*

Then, using S as defined above it is not hard to show the following:

Theorem 2. *If S is a $b \times D$ matrix whose columns are 4-wise independent $\{\pm\frac{1}{\sqrt{b}}\}$ random variables, then*

- $\mathbb{E}[\langle Sx, Sy\rangle] = \langle x, y\rangle$
- $Var(\langle Sx, Sy\rangle) = \frac{2}{k}||x||_2^2||y||_2^2$

for any $x, y \in \mathbb{R}^D$.

This theorem follows a similar set of calculations we did for mean and variance of our original counter. It also implies that S is a good "intermediate" multiplier for matrix multiplication:

Corollary 1. *For S as above, with $b \geq \frac{2}{\epsilon^2\delta}$, then with probability $1-\delta$,*

$$||AB - C||_F \leq \epsilon||A||_F||B||_F$$

where $C = AS^TSB$, which takes only $\left(\frac{n^2}{\epsilon^2\delta} + \frac{n}{\epsilon^2\delta^2}\right)$ time to compute.

2.3 Computing F_0: a lower bound

If one wants to compute F_0, the number of distinct elements we've seen in our stream, we're out of luck if we want an exact solution without randomness, if we're trying to save any space.

Theorem 3. *Suppose a deterministic function outputs the exact value of F_0 for any stream of at most N elements. Then, it must use $\Omega(N)$ space if $N \leq 2|U|$.*

Proof. Consider a stream of $N-1$ unique elements S. Then the function must output $N-1$ on S. Moreover, it must correctly output N for any element in $e \in U \setminus S$ added in the N-th position, but output $N-1$ for any element $e \in S$ added in the Nth position. That means the function must have "memorized" S, which takes

$$\Omega\left(\log\binom{|U|}{N-1}\right) \geq \Omega\left((N-1)\left(\log|U| - \log(|N-1|)\right)\right) = \Omega(N)$$

bits. □

2.4 Computing F_0 with additions only

A good idea for getting some randomized algorithm which approximately answers queries to F_0 will be to use hash functions. Imagine d elements are independently mapped uniformly to $[0, 1]$. One would expect the smallest element to be somewhere near $1/d$ in this mapped space, and the sth smallest element to be near s/d. If we could construct a map which always mapped x to the same point, and mapped all of the elements in our stream to a point on $[0, 1]$, this idea could help us estimate the number of unique elements. Somewhat more formally, consider the algorithm:

Algorithm 2: F_0 estimator without deletions

Pick a 2-wise hash function $h : U \to [m]$;
Keep track of the smallest hashed values amongst $h(a_1), \ldots h(a_t)$;
Return $\frac{m \cdot s}{\text{sth smallest hashed value}}$

The idea behind this algorithm is that it is invariant under adding an element we've already seen. Larger values of s will have smaller variance than $s = 1$, but larger values of s will require us to remember the s smallest hashed elements, so there will be a balancing act between larger and smaller s. The larger m is the better of an approximation we'll get to the unit interval, but the larger the number of bits we'll need to store to remember a 2-wise hash function.

For more details on how to compute the variance, see `https://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15859-s15/www/lectures/lecture20.pdf`.

References

Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. *Journal of Computer and system sciences*, 58(1):137–147, 1999.

Sumit Ganguly. Estimating frequency moments of data streams using random linear combinations. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*, pages 369–380. Springer, 2004.